

Optimasi Kombinasi Barang pada Permasalahan 0/1 Knapsack Menggunakan Algoritma Branch and Bound

Mata Kuliah: Analisa Algoritma
Dosen Pengampu: Ahmad Taufiq Akbar



DISUSUN OLEH :

Reno Miftahudin	123240248
Faiz Maulana Budiaji	123240249
Kevin Prasetya	123240231
Muhammad Garnida B. N.	123240230

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2025/2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Permasalahan optimasi merupakan salah satu bidang penting dalam ilmu komputer yang bertujuan untuk memperoleh solusi terbaik dari sejumlah alternatif dengan tetap memperhatikan batasan tertentu. Berbagai permasalahan optimasi banyak diterapkan dalam kehidupan sehari-hari, seperti pengelolaan sumber daya, penjadwalan, distribusi barang, hingga pemilihan kombinasi barang. Oleh karena itu, diperlukan algoritma yang mampu menghasilkan solusi optimal secara efektif dan efisien.

Salah satu permasalahan optimasi yang sering digunakan sebagai studi kasus adalah 0/1 Knapsack Problem. Permasalahan ini mengharuskan pemilihan kombinasi barang yang memberikan keuntungan maksimum tanpa melebihi kapasitas yang tersedia. Setiap barang hanya memiliki dua pilihan, yaitu diambil atau tidak diambil, sehingga diperlukan proses pencarian kombinasi yang tepat agar diperoleh hasil yang optimal.

Pada studi kasus ini, seorang pencuri berhasil memasuki sebuah toko perhiasan dan menemukan empat barang berharga yang memiliki nilai keuntungan dan berat yang berbeda-beda. Pencuri tersebut hanya memiliki sebuah ransel dengan kapasitas maksimum 15 kg sehingga harus menentukan kombinasi barang yang memberikan keuntungan terbesar tanpa melampaui batas kapasitas ransel. Kondisi tersebut menjadikan permasalahan ini sebagai contoh penerapan 0/1 Knapsack Problem.

Untuk menyelesaikan permasalahan tersebut, digunakan algoritma Branch and Bound. Algoritma ini bekerja dengan membangun pohon pencarian dan menggunakan nilai batas (*bound*) untuk memperkirakan keuntungan maksimum yang dapat diperoleh dari setiap cabang. Cabang yang tidak berpotensi menghasilkan solusi yang lebih baik akan dihentikan proses pencariannya (*pruning*), sehingga jumlah pencarian dapat dikurangi dan solusi optimal dapat ditemukan dengan lebih efisien.

Sebagai pembandingan, laporan ini juga membahas algoritma Backtracking. Algoritma Backtracking mencari solusi dengan menelusuri setiap kemungkinan secara rekursif dan kembali ke langkah sebelumnya apabila suatu kombinasi tidak memenuhi syarat. Perbandingan antara Branch and Bound dan Backtracking dilakukan untuk mengetahui perbedaan mekanisme pencarian, efisiensi, serta kemampuan kedua algoritma dalam menyelesaikan permasalahan 0/1 Knapsack Problem.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam laporan ini adalah sebagai berikut.

1. Bagaimana menerapkan algoritma Branch and Bound untuk menyelesaikan permasalahan 0/1 Knapsack Problem?
2. Bagaimana proses pencarian solusi optimal menggunakan algoritma Branch and Bound pada studi kasus yang diberikan?
3. Bagaimana perbandingan algoritma Branch and Bound dan Backtracking dalam menyelesaikan permasalahan 0/1 Knapsack Problem ditinjau dari mekanisme pencarian dan efisiensi?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah disusun, tujuan dari laporan ini adalah sebagai berikut.

1. Menerapkan algoritma Branch and Bound untuk menyelesaikan permasalahan 0/1 Knapsack Problem.
2. Menjelaskan proses pencarian solusi optimal menggunakan algoritma Branch and Bound pada studi kasus yang diberikan.
3. Membandingkan algoritma Branch and Bound dan Backtracking berdasarkan mekanisme pencarian serta efisiensinya dalam menyelesaikan permasalahan 0/1 Knapsack Problem.

BAB II

DASAR TEORI

2.1 Algoritma Branch and Bound

Branch and Bound merupakan salah satu algoritma yang digunakan untuk menyelesaikan permasalahan optimasi. Algoritma ini bekerja dengan membangun pohon pencarian (*search tree*) dan menggunakan nilai batas (*bound*) untuk memperkirakan solusi terbaik yang masih mungkin diperoleh pada setiap cabang. Cabang yang diperkirakan tidak mampu menghasilkan solusi yang lebih baik dibandingkan solusi sementara akan dihentikan proses pencariannya (*pruning*). Dengan demikian, Branch and Bound dapat mengurangi jumlah pencarian sehingga proses penyelesaian menjadi lebih efisien.

2.2 Konsep Branch and Bound

Proses kerja algoritma Branch and Bound terdiri atas dua tahapan utama, yaitu *branching* dan *bounding*. Branching merupakan proses membentuk cabang-cabang baru dari setiap kemungkinan solusi, sedangkan bounding adalah proses menghitung nilai batas untuk menentukan apakah suatu cabang masih layak dieksplorasi. Apabila nilai batas suatu cabang lebih kecil daripada solusi terbaik yang telah ditemukan, maka cabang tersebut tidak akan diproses lebih lanjut.

2.3 0/1 Knapsack Problem

0/1 Knapsack Problem merupakan permasalahan optimasi yang bertujuan memilih kombinasi barang dengan keuntungan maksimum tanpa melebihi kapasitas yang tersedia. Setiap barang hanya dapat dipilih satu kali atau tidak dipilih sama sekali. Permasalahan ini banyak digunakan sebagai contoh penerapan algoritma optimasi karena memiliki ruang solusi yang cukup besar ketika jumlah barang bertambah.

2.4 Algoritma Backtracking

Backtracking merupakan algoritma pencarian yang bekerja dengan menelusuri seluruh kemungkinan solusi secara rekursif. Algoritma ini akan kembali ke langkah sebelumnya (*backtrack*) apabila solusi yang sedang dibangun tidak memenuhi syarat atau tidak memungkinkan menghasilkan solusi yang diinginkan. Berbeda dengan Branch and Bound, Backtracking tidak menggunakan estimasi nilai batas sehingga jumlah cabang yang dieksplorasi umumnya lebih banyak.

2.5 Perbandingan Branch and Bound dan Backtracking

Meskipun sama-sama menggunakan konsep pohon pencarian, Branch and Bound dan Backtracking memiliki mekanisme yang berbeda. Branch and Bound menggunakan nilai *bound* untuk memangkas cabang yang tidak berpotensi menghasilkan solusi optimal sehingga lebih efisien pada permasalahan optimasi. Sebaliknya, Backtracking hanya melakukan

pemangkasan berdasarkan pelanggaran terhadap suatu kendala (*constraint*) sehingga lebih sesuai digunakan untuk permasalahan pencarian solusi yang harus memenuhi aturan tertentu.

BAB III PEMBAHASAN

3.1 Analisis Studi Kasus

Studi kasus yang digunakan dalam laporan ini adalah 0/1 Knapsack Problem, yaitu permasalahan optimasi dalam menentukan kombinasi barang yang memberikan keuntungan maksimum tanpa melebihi kapasitas yang tersedia. Pada kasus ini, seorang pencuri berhasil memasuki sebuah toko perhiasan dan menemukan empat barang berharga yang memiliki nilai keuntungan (profit) dan berat (weight) yang berbeda. Pencuri tersebut hanya memiliki sebuah ransel dengan kapasitas maksimum 15 kg, sehingga harus memilih kombinasi barang yang paling menguntungkan. Data barang yang digunakan pada studi kasus ditunjukkan pada Tabel 3.1.

Tabel 3.1 Data Barang

Barang	Profit (Juta Rupiah)	Berat (kg)
Barang 1	10	2
Barang 2	10	4
Barang 3	12	6
Barang 4	18	9

Kapasitas maksimum ransel (W) adalah 15 kg. Berdasarkan data tersebut, tujuan yang ingin dicapai adalah memperoleh kombinasi barang dengan total keuntungan terbesar tanpa melebihi kapasitas maksimum ransel. Karena setiap barang hanya dapat dipilih satu kali atau tidak dipilih sama sekali, maka permasalahan ini termasuk ke dalam 0/1 Knapsack Problem.

3.2 Penyelesaian Secara Manual Menggunakan Branch and Bound

Algoritma Branch and Bound merupakan algoritma optimasi yang bekerja dengan membangun pohon pencarian (*State Space Tree*). Setiap simpul (*node*) pada pohon menyatakan suatu keadaan (*state*) yang berisi keputusan untuk mengambil atau tidak mengambil suatu barang. Berbeda dengan metode pencarian menyeluruh (*brute force*), Branch and Bound menggunakan nilai bound sebagai estimasi keuntungan maksimum yang masih mungkin diperoleh dari suatu cabang. Nilai tersebut digunakan untuk menentukan apakah suatu cabang masih layak dieksplorasi atau dapat dipangkas (*pruning*).

Sebelum proses pencarian dimulai, algoritma mengurutkan barang berdasarkan rasio profit/weight dari nilai terbesar ke nilai terkecil.

Pengurutan ini bertujuan untuk memperoleh nilai bound yang lebih akurat sehingga proses pencarian menjadi lebih efisien.

Selanjutnya algoritma membentuk simpul awal (root node) yang belum memilih barang apa pun. Dari simpul tersebut akan dibentuk dua cabang, yaitu memilih barang dan tidak memilih barang. Pada setiap simpul dihitung total berat, total keuntungan, serta nilai bound. Node yang memiliki nilai bound terbesar akan dipilih terlebih dahulu untuk dikembangkan menggunakan Priority Queue. Proses pencarian secara manual ditunjukkan pada Tabel 3.2.

Tabel 3.2 Proses Branch and Bound

Langka h	Node	Barang	Berat (kg)	Bound (Juta)	Profit (Juta)	Keputusa n
1	Root	-	0	38	0	Membent uk node awal
2	N1	B1	2	38	10	Node dikemban gkan
3	N2	B1 + B2	6	38	20	Node dikemban gkan
4	N3	B1 + B2 + B3	12	38	32	Node dikemban gkan
5	N4	B1 + B2 + B3 + B4	21	0	50	Dipangka s (berat > 15 kg)
6	N5	B1 + B2 + B4	15	38	38	Solusi Optimal
7	N6	B1 + B3 + B4	17	0	40	Dipangka s (berat > 15 kg)
8	N7	B2 + B3 + B4	19	0	40	Dipangka s (berat > 15 kg)
9	N8	Kombinas i lainnya	-	-	-	Tidak memberik

						an profit lebih besar
--	--	--	--	--	--	-----------------------

Berdasarkan proses tersebut, algoritma tidak perlu mengeksplorasi seluruh kemungkinan kombinasi barang. Beberapa cabang dihentikan lebih awal karena telah diketahui tidak memenuhi batas kapasitas atau diperkirakan tidak mampu menghasilkan keuntungan yang lebih besar daripada solusi terbaik yang telah ditemukan.

3.3 Penyelesaian Secara Manual Menggunakan Backtracking

Algoritma Backtracking merupakan algoritma pencarian yang bekerja secara rekursif dengan mengeksplorasi setiap kemungkinan solusi yang dapat dibentuk. Pada setiap langkah, algoritma memiliki dua pilihan terhadap setiap barang, yaitu mengambil atau tidak mengambil barang tersebut. Jika suatu kombinasi tidak memenuhi syarat, yaitu total berat melebihi kapasitas maksimum ransel, maka algoritma akan kembali (*backtrack*) ke langkah sebelumnya untuk mencoba kombinasi yang lain.

Algoritma memulai pencarian dari keadaan awal (*root*) tanpa memilih barang apa pun. Selanjutnya algoritma akan mencoba mengambil Barang 1, kemudian Barang 2, Barang 3, dan Barang 4 secara berurutan. Apabila total berat melebihi kapasitas maksimum, algoritma akan melakukan proses backtrack, yaitu kembali ke node sebelumnya dan mencoba cabang yang lain hingga seluruh kemungkinan kombinasi selesai diperiksa. Proses pencarian secara manual ditunjukkan pada Tabel 3.3.

Tabel 3.3 Proses Pencarian Backtracking

Langkah	Kombinasi	Berat (kg)	Profit (juta)	Keputusan
1	-	0	0	Mulai
2	B1	2	10	Lanjut
3	B1+B2	6	20	Lanjut
4	B1+B2+B3	12	32	Lanjut
5	B1+B2+B3+B ₄	21	50	Backtrack
6	B1+B2+B4	15	38	Solusi Terbaik
7	B1+B3+B4	17	40	Backtrack

8	B2+B3+B4	19	40	Backtrack
9	Kombinasi lainnya	-	-	Dievaluasi

Berdasarkan proses tersebut, algoritma Backtracking berhasil menemukan kombinasi optimal berupa Barang 1, Barang 2, dan Barang 4 dengan total berat 15 kg dan total keuntungan Rp38 juta. Berbeda dengan Branch and Bound, algoritma Backtracking harus menelusuri seluruh kemungkinan kombinasi yang memenuhi aturan sebelum memastikan bahwa solusi yang diperoleh merupakan solusi optimal.

3.4 Analisis Kompleksitas

3.4.1 Analisis Kompleksitas Branch and Bound

Kompleksitas waktu (*Time Complexity*) algoritma Branch and Bound pada permasalahan 0/1 Knapsack berada pada kondisi terburuk sebesar $O(2^n)$, dengan n merupakan jumlah barang. Hal ini disebabkan pada kondisi tertentu seluruh kemungkinan kombinasi masih dapat dieksplorasi.

Namun, pada praktiknya Branch and Bound memiliki performa yang lebih baik dibandingkan pencarian menyeluruh karena menggunakan proses bounding untuk menghilangkan cabang yang diperkirakan tidak menghasilkan solusi optimal. Dengan demikian, jumlah node yang diproses menjadi lebih sedikit sehingga waktu eksekusi menjadi lebih efisien.

Kompleksitas ruang (*Space Complexity*) dipengaruhi oleh jumlah node yang disimpan dalam Priority Queue. Pada kondisi terburuk kompleksitas ruang dapat mencapai $O(2^n)$, tetapi pada banyak kasus nyata penggunaan memori lebih kecil karena adanya proses pruning yang mengurangi jumlah node yang perlu disimpan.

3.4.2 Analisis Kompleksitas BackTracking

Pada algoritma Backtracking, setiap barang memiliki dua pilihan, yaitu diambil atau tidak diambil. Oleh karena itu, jumlah kemungkinan kombinasi yang harus diperiksa adalah 2^n , dengan n merupakan jumlah barang.

Untuk studi kasus ini terdapat empat barang sehingga jumlah kombinasi yang mungkin adalah: $2^4 = 16$ kombinasi

Kompleksitas waktu (*Time Complexity*) algoritma Backtracking adalah $O(2^n)$ karena pada kondisi terburuk algoritma akan mengeksplorasi seluruh kemungkinan kombinasi.

Kompleksitas ruang (*Space Complexity*) adalah $O(n)$ karena algoritma menggunakan rekursi dan kedalaman maksimum sama dengan jumlah barang yang diproses.

3.5 Hasil Analisis

3.5.1 Hasil Algoritma Branch and Bound

Berdasarkan hasil penyelesaian menggunakan algoritma Branch and Bound, diperoleh kombinasi barang yang memberikan keuntungan maksimum tanpa melampaui kapasitas ransel sebesar 15 kg. Dari seluruh kemungkinan kombinasi yang dievaluasi, kombinasi terbaik adalah Barang 1, Barang 2, dan Barang 4 dengan total berat 15 kg dan total keuntungan sebesar Rp38 juta.

Algoritma Branch and Bound bekerja dengan membangun pohon pencarian (State Space Tree) dan menghitung nilai bound pada setiap simpul. Nilai bound digunakan untuk memperkirakan keuntungan maksimum yang masih mungkin diperoleh dari suatu cabang. Apabila nilai bound suatu cabang lebih kecil daripada keuntungan terbaik yang telah ditemukan, maka cabang tersebut tidak akan diproses lebih lanjut (pruning). Mekanisme ini membuat algoritma tidak perlu mengeksplorasi seluruh kemungkinan kombinasi sehingga proses pencarian menjadi lebih efisien.

Pada studi kasus ini, beberapa kombinasi barang yang memiliki total berat melebihi kapasitas ransel, seperti kombinasi Barang 1 + Barang 3 + Barang 4, Barang 2 + Barang 3 + Barang 4, dan Barang 1 + Barang 2 + Barang 3 + Barang 4, secara otomatis dieliminasi karena tidak memenuhi batas kapasitas. Selain itu, cabang yang diperkirakan tidak mampu menghasilkan keuntungan lebih besar daripada solusi sementara juga dipangkas berdasarkan nilai bound.

Hasil analisis menunjukkan bahwa algoritma Branch and Bound berhasil menemukan solusi optimal dengan proses pencarian yang lebih efisien dibandingkan pencarian secara menyeluruh (brute force). Meskipun pada kondisi terburuk algoritma memiliki kompleksitas waktu $O(2^n)$, penggunaan teknik bounding dan pruning mampu mengurangi jumlah simpul yang dieksplorasi. Oleh karena itu, algoritma Branch and Bound sangat sesuai digunakan untuk menyelesaikan permasalahan optimasi seperti 0/1 Knapsack Problem, terutama ketika jumlah barang yang diproses semakin banyak.

3.5.2 Hasil Analisis Backtracking

Berdasarkan hasil penyelesaian menggunakan algoritma Backtracking, diperoleh kombinasi barang yang memberikan keuntungan maksimum tanpa melebihi kapasitas ransel sebesar 15 kg. Dari seluruh kombinasi yang dieksplorasi, kombinasi terbaik adalah Barang 1, Barang 2, dan Barang 4 dengan total berat 15 kg dan total keuntungan sebesar Rp38 juta.

Algoritma Backtracking bekerja dengan menelusuri setiap kemungkinan kombinasi barang secara rekursif. Pada setiap langkah, algoritma akan memutuskan apakah suatu barang diambil atau tidak diambil. Apabila kombinasi yang terbentuk memiliki total berat melebihi kapasitas maksimum, algoritma akan kembali (backtrack) ke langkah sebelumnya untuk mencoba kombinasi yang lain. Proses ini dilakukan hingga seluruh kemungkinan kombinasi telah diperiksa.

Pada studi kasus ini terdapat empat barang sehingga jumlah kemungkinan kombinasi yang dapat dibentuk adalah $2^4 = 16$ kombinasi. Algoritma Backtracking mengevaluasi setiap kombinasi yang mungkin untuk memastikan bahwa solusi yang diperoleh benar-benar merupakan solusi optimal. Kombinasi yang memiliki berat melebihi 15 kg dinyatakan tidak valid dan tidak dipertimbangkan sebagai solusi.

Hasil analisis menunjukkan bahwa algoritma Backtracking berhasil menemukan solusi optimal dengan kombinasi Barang 1, Barang 2, dan Barang 4. Namun, untuk memperoleh solusi tersebut algoritma harus mengeksplorasi lebih banyak kemungkinan dibandingkan Branch and Bound karena tidak menggunakan proses bounding untuk memangkas cabang pencarian. Meskipun demikian, Backtracking tetap mampu memberikan solusi yang optimal sehingga sesuai digunakan untuk menyelesaikan permasalahan 0/1 Knapsack Problem, terutama ketika jumlah data yang diproses relatif sedikit.

3.6 Perbandingan Algoritma Branch and Bound dan Backtracking

Setelah dilakukan implementasi dan pengujian terhadap kedua algoritma pada studi kasus 0/1 Knapsack Problem, diperoleh hasil bahwa algoritma Branch and Bound maupun Backtracking mampu menemukan solusi optimal yang sama, yaitu memilih Barang 1, Barang 2, dan Barang 4 dengan total keuntungan sebesar Rp38 juta dan total berat 15 kg. Perbedaan utama kedua algoritma terletak pada mekanisme pencarian solusi dan efisiensi proses pencariannya.

Tabel 3.4 menunjukkan perbandingan karakteristik algoritma Branch and Bound dan Backtracking.

Tabel 3.4 Perbandingan Branch and Bound dan Backtracking

Aspek	Branch and Bound	Backtracking
Tujuan	Mencari solusi optimal dengan proses pencarian yang efisien	Mencari solusi optimal dengan menelusuri seluruh kemungkinan
Konsep Dasar	Menggunakan teknik branching, bounding,	Menggunakan rekursi dan proses backtrack

	dan pruning	
Mekanisme Pencarian	Memilih node yang paling menjanjikan berdasarkan nilai bound	Menelusuri setiap kemungkinan secara rekursif
Teknik Pruning	Berdasarkan nilai bound dan kapasitas	Berdasarkan pelanggaran kapasitas saja
Struktur Data	Priority Queue	Rekursi (Call Stack)
Jumlah Node yang Dieksplorasi	Lebih sedikit	Lebih banyak
Kompleksitas Waktu (Worst Case)	$O(2^n)$	$O(2^n)$
Kompleksitas Ruang	$O(2^n)$	$O(n)$
Solusi yang Diperoleh	Barang 1, Barang 2, dan Barang 4	Barang 1, Barang 2, dan Barang 4
Total Profit	Rp38 juta	Rp38 juta
Total Berat	15 kg	15 kg
Efisiensi Pencarian	Lebih efisien karena menggunakan nilai bound	Kurang efisien karena harus mengeksplorasi lebih banyak kombinasi
Kelebihan	Mempercepat pencarian solusi optimal dengan mengurangi jumlah node yang diproses	Implementasi lebih sederhana dan mudah dipahami
Kekurangan	Perhitungan bound lebih kompleks	Membutuhkan waktu lebih lama ketika jumlah data semakin besar

Berdasarkan Tabel 3.4 dapat diketahui bahwa kedua algoritma menghasilkan solusi optimal yang sama, yaitu memilih Barang 1, Barang 2, dan Barang 4 dengan total keuntungan sebesar Rp38 juta dan total berat 15 kg. Perbedaan utama terletak pada proses pencarian solusi. Algoritma Branch and Bound mampu mengurangi jumlah node yang dieksplorasi

melalui proses bounding dan pruning, sehingga pencarian menjadi lebih efisien. Sebaliknya, algoritma Backtracking harus mengevaluasi lebih banyak kemungkinan karena tidak menggunakan estimasi nilai bound, sehingga waktu pencarian cenderung lebih lama meskipun tetap menghasilkan solusi yang optimal.

BAB IV

IMPLEMENTASI PROGRAM

4.1 Penjelasan Program

Program pada penelitian ini diimplementasikan menggunakan bahasa pemrograman C++. Implementasi dilakukan dengan menggabungkan algoritma Branch and Bound dan Backtracking ke dalam satu berkas program (main.cpp). Penggabungan kedua algoritma dalam satu program bertujuan agar proses pengujian dilakukan pada lingkungan kompilasi, perangkat keras, dan data uji yang sama, sehingga hasil perbandingan waktu eksekusi menjadi lebih adil (*fair comparison*).

Data masukan yang digunakan berupa empat buah barang yang masing-masing memiliki nilai keuntungan (*profit*) dan berat (*weight*), dengan kapasitas maksimum ransel sebesar 15 kg. Program akan menjalankan algoritma Branch and Bound terlebih dahulu untuk memperoleh solusi optimal beserta waktu eksekusinya. Selanjutnya, data yang sama diproses menggunakan algoritma Backtracking sehingga hasil kedua algoritma dapat dibandingkan secara langsung.

Implementasi Branch and Bound memanfaatkan struktur data Priority Queue untuk memilih node dengan nilai bound terbesar. Setiap node menyimpan informasi mengenai level, berat, keuntungan, serta nilai bound yang digunakan untuk menentukan apakah suatu cabang masih layak dieksplorasi. Cabang yang tidak berpotensi menghasilkan solusi yang lebih baik akan dipangkas (*pruning*), sehingga proses pencarian menjadi lebih efisien.

Sementara itu, implementasi Backtracking menggunakan pendekatan rekursif. Pada setiap pemanggilan fungsi, program menentukan dua kemungkinan keputusan, yaitu mengambil atau tidak mengambil suatu barang. Apabila total berat telah melebihi kapasitas maksimum, fungsi akan kembali (*backtrack*) ke kondisi sebelumnya dan melanjutkan pencarian pada cabang yang lain hingga seluruh kemungkinan kombinasi selesai diperiksa.

Untuk mengukur performa kedua algoritma, program memanfaatkan pustaka <chrono> yang tersedia pada C++. Waktu eksekusi dihitung dalam satuan microsecond (μs) dengan mencatat waktu sebelum algoritma dijalankan dan setelah algoritma selesai memperoleh solusi. Hasil pengukuran tersebut kemudian digunakan sebagai dasar perbandingan efisiensi antara algoritma Branch and Bound dan Backtracking.

4.2 Program Branch and Bound

```

#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

using namespace std;

// Struktur data untuk menampung spesifikasi barang
struct Item {
    int id;
    int value;
    int weight;
    double ratio; // Rasio Keuntungan / Berat
};

// Struktur data untuk Node pada Pohon Ruang Status (State Space Tree)
struct Node {
    int level; // Indeks level barang yang sedang diproses
    int profit; // Akumulasi keuntungan saat ini
    int weight; // Akumulasi berat saat ini
    double bound; // Prediksi batas atas keuntungan maksimal dari cabang ini
};

// Fungsi pembandingan untuk mengurutkan barang berdasarkan rasio terbesar ke terkecil
bool compareItems(Item a, Item b) {
    return a.ratio > b.ratio;
}

// Fungsi pembandingan untuk Antrean Prioritas (Mendahulukan Node dengan Bound terbesar)
struct CompareNode {
    bool operator()(Node const& n1, Node const& n2) {
        return n1.bound < n2.bound;
    }
};

// Fungsi menghitung Upper Bound (Estimasi keuntungan menggunakan Fractional Knapsack)
double calculateBound(Node u, int n, int capacity, const vector<Item>& items) {
    if (u.weight >= capacity) return 0;

    double profit_bound = u.profit;
    int j = u.level + 1;
    int total_weight = u.weight;

    // Masukkan barang secara utuh selama kapasitas ransel masih muat

```

```

        while (j < n && total_weight + items[j].weight <=
capacity) {
            total_weight += items[j].weight;
            profit_bound += items[j].value;
            j++;
        }

        // Jika masih ada sisa kapasitas, ambil pecahan nilai
dari barang berikutnya
        if (j < n) {
            profit_bound += (capacity - total_weight) *
items[j].ratio;
        }

        return profit_bound;
    }

int knapsackBranchAndBound(int capacity, vector<Item>&
items) {
    int n = items.size();

    // Syarat wajib Branch & Bound: Urutkan barang
berdasarkan rasio profit/weight secara menurun
    sort(items.begin(), items.end(), compareItems);

    priority_queue<Node, vector<Node>, CompareNode> pq;
    Node u, v;

    // Inisialisasi Node Akar (Root) awal
    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = calculateBound(u, n, capacity, items);
    pq.push(u);

    int max_profit = 0;

    while (!pq.empty()) {
        // Ambil Node yang memiliki nilai prediksi
keuntungan (bound) tertinggi
        u = pq.top();
        pq.pop();

        // Pangkas cabang (Pruning) jika prediksi nilainya
sudah tidak lebih baik dari max_profit saat ini
        if (u.bound <= max_profit) continue;

        // Bergeser ke level barang berikutnya
        v.level = u.level + 1;

        // JALUR KIRI: Keputusan memuat/mengambil barang
ke dalam ransel

```



```

        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

        // Jika berat aman dan profitnya memecahkan rekor
        baru, perbarui max_profit
        if (v.weight <= capacity && v.profit > max_profit)
        {
            max_profit = v.profit;
        }

        v.bound = calculateBound(v, n, capacity, items);
        // Jika cabangnya masih potensial menghasilkan
        nilai lebih tinggi, masukkan ke antrean
        if (v.bound > max_profit) {
            pq.push(v);
        }

        // JALUR KANAN: Keputusan TIDAK mengambil barang
        ke dalam ransel
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = calculateBound(v, n, capacity, items);

        if (v.bound > max_profit) {
            pq.push(v);
        }
    }

    return max_profit;
}

int main() {
    // Input Data Sesuai Studi Kasus Toko Perhiasan
    vector<int> values = {10, 10, 12, 18};
    vector<int> weights = {2, 4, 6, 9};
    int capacity = 15;
    int n = values.size();

    vector<Item> items(n);
    for (int i = 0; i < n; i++) {
        items[i].id = i + 1;
        items[i].value = values[i];
        items[i].weight = weights[i];
        items[i].ratio = (double)values[i] / weights[i];
    }

    // Menjalankan Fungsi Branch and Bound
    int max_value = knapsackBranchAndBound(capacity,
    items);

    // Menampilkan Hasil Output
    cout << "=====

```

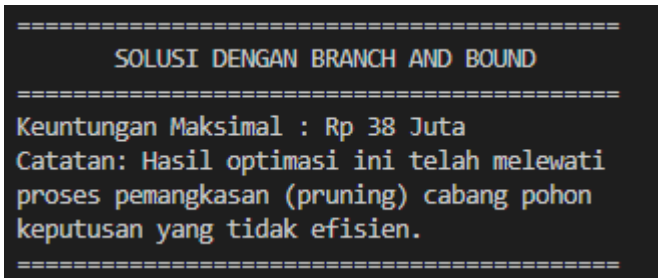
```

<< endl;
    cout << "          SOLUSI DENGAN BRANCH AND BOUND          "
<< endl;
    cout << "===== "
<< endl;
    cout << "Keuntungan Maksimal : Rp " << max_value << "
Juta" << endl;
    cout << "Catatan: Hasil optimasi ini telah melewati"
<< endl;
    cout << "proses pemangkasan (pruning) cabang pohon" <<
endl;
    cout << "keputusan yang tidak efisien." << endl;
    cout << "===== "
<< endl;

    return 0;
}

```

4.2.1 Hasil Pengujian Program



```

=====
          SOLUSI DENGAN BRANCH AND BOUND
=====
Keuntungan Maksimal : Rp 38 Juta
Catatan: Hasil optimasi ini telah melewati
proses pemangkasan (pruning) cabang pohon
keputusan yang tidak efisien.
=====

```

4.2.2 Analisis Hasil

Berdasarkan hasil implementasi, algoritma Branch and Bound berhasil menemukan kombinasi barang yang memberikan keuntungan maksimum sebesar Rp38 juta dengan memilih Barang 1, Barang 2, dan Barang 4 serta total berat 15 kg. Algoritma bekerja dengan membangun pohon pencarian dan menggunakan nilai bound untuk memperkirakan keuntungan maksimum yang dapat diperoleh pada setiap cabang. Node yang memiliki peluang terbaik diproses terlebih dahulu, sedangkan cabang yang tidak berpotensi menghasilkan solusi optimal akan dipangkas (*pruning*).

Penggunaan teknik bounding dan pruning membuat algoritma Branch and Bound lebih efisien dibandingkan pencarian secara menyeluruh karena tidak semua kemungkinan kombinasi perlu dieksplorasi. Meskipun memiliki kompleksitas waktu terburuk $O(2^n)$, pada implementasi ini algoritma mampu mengurangi jumlah node yang diproses sehingga solusi optimal dapat diperoleh dengan waktu pencarian yang lebih cepat.

4.3 Program Backtracking

```
#include <iostream>
```

```

#include <vector>

using namespace std;

// Variabel global untuk menyimpan solusi keuntungan tertinggi dan kombinasi terbaik
int max_value = 0;
vector<int> best_combination;

// Fungsi rekursif Backtracking
void knapsackBacktracking(const vector<int>& weights,
                        const vector<int>& values, int capacity,
                        int n, int index, int
current_weight, int current_value, vector<int>&
current_choices) {

    // Base Case: Jika semua barang sudah dipertimbangkan
    (mencapai ujung percabangan)
    if (index == n) {
        // Jika keuntungan saat ini lebih besar dari yang
        pernah dicatat, perbarui nilai terbaik
        if (current_value > max_value) {
            max_value = current_value;
            best_combination = current_choices;
        }
        return;
    }

    // Opsi 1: AMBIL barang saat ini (jika kapasitas
    ransel masih muat)
    if (current_weight + weights[index] <= capacity) {
        current_choices[index] = 1; // Tandai barang
        diambil
        knapsackBacktracking(weights, values, capacity, n,
        index + 1,
                                current_weight +
weights[index],
                                current_value +
values[index], current_choices);
    }

    // Opsi 2: JANGAN AMBIL barang saat ini (Proses
    Backtrack)
    current_choices[index] = 0; // Tandai barang tidak
    diambil
    knapsackBacktracking(weights, values, capacity, n,
    index + 1,
                                current_weight, current_value,
current_choices);
}

int main() {

```

```

        // Input Data Sesuai Studi Kasus Toko Perhiasan
        vector<int> values = {10, 10, 12, 18}; // Keuntungan
(dalam Juta Rp)
        vector<int> weights = {2, 4, 6, 9};    // Berat barang
(dalam kg)
        int capacity = 15;                      // Kapasitas
maksimal ransel (kg)
        int n = values.size();

        vector<int> current_choices(n, 0);
        best_combination.resize(n, 0);

        // Menjalankan Fungsi Backtracking dimulai dari indeks
barang ke-0
        knapsackBacktracking(weights, values, capacity, n, 0,
0, 0, current_choices);

        // Menampilkan Hasil Output
        cout << "=====
<< endl;
        cout << "          SOLUSI DENGAN BACKTRACKING          "
<< endl;
        cout << "=====
<< endl;
        cout << "Keuntungan Maksimal : Rp " << max_value << "
Juta" << endl;
        cout << "Kombinasi Barang      : " << endl;

        int total_berat = 0;
        for (int i = 0; i < n; i++) {
            if (best_combination[i] == 1) {
                cout << "- Barang " << (i + 1) << " (Berat: "
<< weights[i] << " kg, Profit: Rp " << values[i] << "
Juta)" << endl;
                total_berat += weights[i];
            }
        }
        cout << "Total Berat Ransel  : " << total_berat << "
kg" << endl;
        cout << "=====
<< endl;

        return 0;
    }

```

4.3.1 Hasil Pengujian Program

```
=====
SOLUSI DENGAN BACKTRACKING
=====
Keuntungan Maksimal : Rp 38 Juta
Kombinasi Barang :
- Barang 1 (Berat: 2 kg, Profit: Rp 10 Juta)
- Barang 2 (Berat: 4 kg, Profit: Rp 10 Juta)
- Barang 4 (Berat: 9 kg, Profit: Rp 18 Juta)
Total Berat Ransel : 15 kg
=====
```

4.3.2 Analisis Hasil

Berdasarkan hasil implementasi, algoritma Backtracking berhasil menemukan kombinasi barang yang memberikan keuntungan maksimum sebesar Rp38 juta dengan memilih Barang 1, Barang 2, dan Barang 4 serta total berat 15 kg. Algoritma bekerja dengan menelusuri setiap kemungkinan kombinasi barang secara rekursif. Apabila kombinasi yang terbentuk melebihi kapasitas maksimum ransel, algoritma akan kembali (*backtrack*) ke langkah sebelumnya dan mencoba kombinasi yang lain hingga seluruh kemungkinan selesai diperiksa.

Pendekatan tersebut menjamin bahwa solusi yang diperoleh merupakan solusi optimal, namun proses pencariannya memerlukan eksplorasi lebih banyak node dibandingkan Branch and Bound karena tidak menggunakan teknik bounding untuk memangkas cabang yang kurang menjanjikan. Akibatnya, waktu eksekusi cenderung lebih lama ketika jumlah data bertambah, meskipun pada studi kasus ini algoritma tetap mampu menghasilkan solusi yang optimal.

BAB V

ANALISIS KOMPLEKSITAS

5.1 Kompleksitas Algoritma Branch and Bound

Kompleksitas waktu (Time Complexity) algoritma Branch and Bound secara teoritis juga berada pada $O(2^n)$ pada kondisi terburuk. Namun, dalam implementasinya algoritma mampu mengurangi jumlah node yang dieksplorasi melalui proses bounding dan pruning, sehingga rata-rata waktu eksekusinya lebih cepat dibandingkan Backtracking.

Kompleksitas ruang (Space Complexity) algoritma Branch and Bound adalah $O(2^n)$ karena algoritma menyimpan node-node yang masih aktif di dalam Priority Queue. Jumlah node yang disimpan dapat bertambah seiring dengan banyaknya cabang yang masih berpotensi menghasilkan solusi optimal, sehingga kebutuhan memorinya lebih besar dibandingkan algoritma Backtracking.

5.2 Kompleksitas Algoritma Backtracking

Kompleksitas waktu (Time Complexity) algoritma Backtracking adalah $O(2^n)$. Hal ini disebabkan setiap barang memiliki dua kemungkinan keputusan, yaitu diambil atau tidak diambil. Pada kondisi terburuk (worst case), algoritma harus mengeksplorasi seluruh kemungkinan kombinasi untuk memastikan bahwa solusi yang diperoleh merupakan solusi optimal.

Kompleksitas ruang (Space Complexity) algoritma Backtracking adalah $O(n)$. Hal ini karena algoritma menggunakan pendekatan rekursif (Depth-First Search/DFS), sehingga memori yang digunakan hanya sebesar kedalaman rekursi atau sebanyak jumlah barang yang diproses.

5.3 Analisis Perbandingan Kompleksitas

Berdasarkan analisis kompleksitas, kedua algoritma memiliki kompleksitas waktu terburuk yang sama, yaitu $O(2^n)$. Perbedaannya terletak pada mekanisme pencarian solusi. Branch and Bound memanfaatkan proses bounding dan pruning sehingga mampu mengurangi jumlah node yang dieksplorasi dan menghasilkan waktu eksekusi yang lebih efisien. Sebaliknya, Backtracking menggunakan pendekatan Depth-First Search (DFS) yang lebih hemat memori dengan kompleksitas ruang $O(n)$, tetapi harus mengevaluasi lebih banyak kemungkinan kombinasi sebelum memperoleh solusi optimal.

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil analisis, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa algoritma Branch and Bound dan Backtracking mampu menyelesaikan permasalahan 0/1 Knapsack Problem dengan menghasilkan solusi optimal yang sama. Pada studi kasus yang digunakan, kedua algoritma berhasil menentukan kombinasi Barang 1, Barang 2, dan Barang 4 sebagai solusi terbaik dengan total keuntungan sebesar Rp38 juta dan total berat 15 kg, sehingga memenuhi batas kapasitas maksimum ransel.

Meskipun menghasilkan solusi yang sama, kedua algoritma memiliki perbedaan dalam mekanisme pencarian solusi. Algoritma Branch and Bound menggunakan teknik bounding dan pruning untuk mengurangi jumlah cabang yang perlu dieksplorasi sehingga proses pencarian menjadi lebih efisien. Sementara itu, algoritma Backtracking bekerja dengan menelusuri setiap kemungkinan kombinasi secara rekursif dan melakukan backtrack ketika suatu kombinasi tidak memenuhi syarat, sehingga jumlah node yang dieksplorasi lebih banyak.

Berdasarkan hasil implementasi dan analisis kompleksitas, algoritma Branch and Bound menunjukkan performa yang lebih baik dalam hal efisiensi waktu pencarian dibandingkan algoritma Backtracking, terutama ketika jumlah data semakin besar. Di sisi lain, algoritma Backtracking memiliki implementasi yang lebih sederhana dan penggunaan memori yang lebih hemat karena menggunakan pendekatan rekursif (Depth-First Search). Oleh karena itu, pemilihan algoritma dapat disesuaikan dengan karakteristik permasalahan dan kebutuhan sistem. Untuk permasalahan optimasi seperti 0/1 Knapsack Problem, algoritma Branch and Bound merupakan pilihan yang lebih efektif karena mampu memperoleh solusi optimal dengan proses pencarian yang lebih efisien.

6.2 Saran

Penelitian ini masih menggunakan jumlah data yang relatif sedikit, yaitu empat buah barang. Pada penelitian selanjutnya disarankan untuk menggunakan jumlah data yang lebih besar agar perbedaan performa antara algoritma Branch and Bound dan Backtracking dapat terlihat lebih jelas.

Selain itu, penelitian selanjutnya dapat membandingkan kedua algoritma dengan algoritma optimasi lainnya, seperti Dynamic Programming, Greedy, atau Genetic Algorithm, sehingga dapat diketahui algoritma yang paling sesuai untuk berbagai karakteristik permasalahan 0/1 Knapsack Problem.

DAFTAR PUSTAKA

- Agusfrianto, F. A., & Rachmawati, R. (2022). *Perbandingan Metode Branch and Bound dan Enumerasi implisit dalam menyelesaikan masalah Knapsack* (Vol. 13, Number 1).
- Evangelis Purba, M., & Ginting, R. J. E. (n.d.). *JUSINFO (Jurnal Sains dan Informatika) Analisis Efektivitas Metode Branch And Bound Dalam Penyelesaian Masalah Knapsack* (Vol. 1, Number 1).
- Safitri, E., & Najmi, H. (n.d.). *Penerapan Metode Branch and Bound dalam Optimalisasi Produk Mebel (Studi kasus: Toko Mebel di Jalan Marsan Panam)*.

Peranan dan Kontribusi Anggota Kelompok

Nama Anggota	NIM	Deskripsi Tugas & Kontribusi
Reno Miftahudin	123240248	Bertanggung jawab dalam pembuatan dan implementasi source code algoritma Branch and Bound dan Backtracking, serta membantu proses pengujian program.
Faiz Maulana Budiaji	123240249	Bertanggung jawab dalam penyusunan laporan secara keseluruhan, meliputi BAB I sampai BAB VI, analisis studi kasus, penyelesaian manual, analisis kompleksitas, implementasi program, analisis hasil, perbandingan algoritma, serta penyusunan kesimpulan dan saran.
Kevin Prasetya	123240231	Membantu pembuatan dan pengujian source code, melakukan debugging program, serta memastikan hasil implementasi berjalan sesuai dengan algoritma yang digunakan.
Muhammad Garnida B. N.	123240230	Bertanggung jawab dalam penyusunan BAB IV, BAB V, dan BAB VI, meliputi implementasi program, analisis hasil, kesimpulan, saran, serta membantu penyuntingan dan finalisasi laporan.